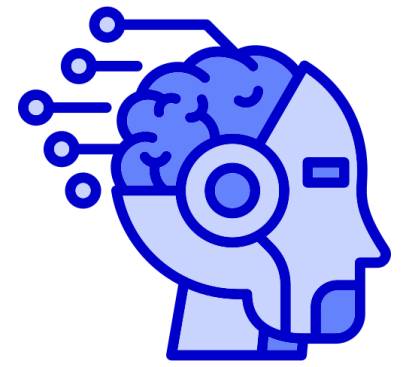
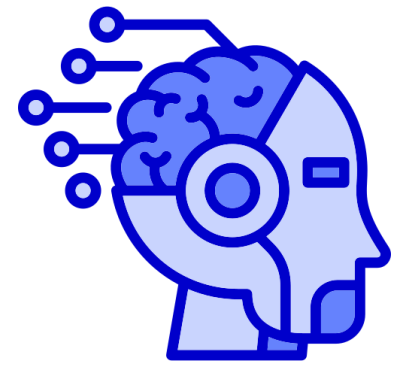




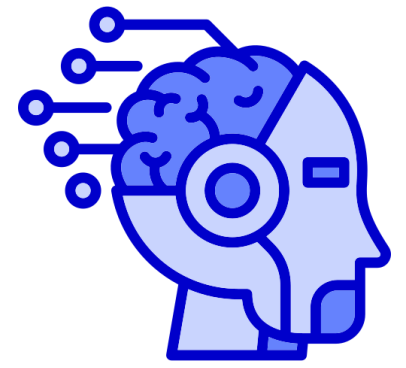
Local Search

Hill Climbing Search



Local Search Algorithms

- In many optimization problems, the **path to the goal is irrelevant**; the *goal state itself is the solution*.
- For example, in the **8-queens problem** what matters is the final configuration of queens, not the order in which they are added.

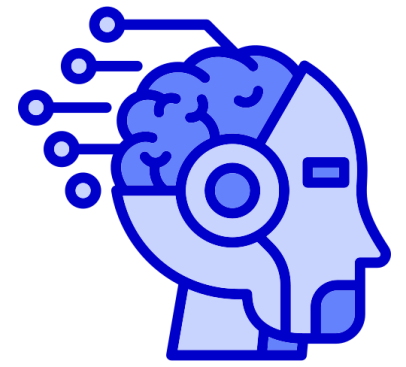


Local Search Algorithms

- State space = set of "complete" configurations

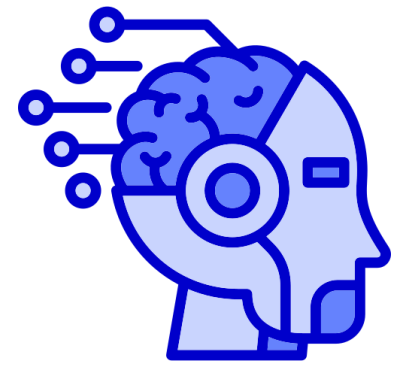
Local search does not **build solution sequentially**; it starts **with complete state** and **try to improve it**.
i.e. n-queen board, All queens are placed.

- Local search algorithms work by iteratively improving the candidate solution by making small changes and evaluating the objective function after each change.



Local Search Algorithms

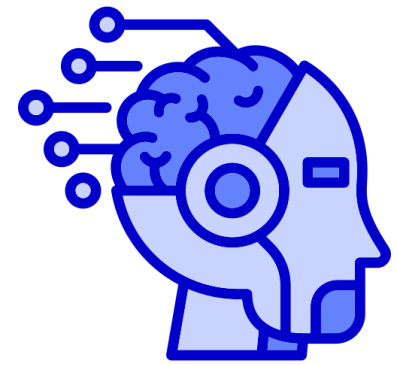
- Local search algorithms operate *by searching from a **start state to neighboring states***, without keeping track of the paths, **nor the set of states that have been reached**.
- **Have Advantages of:**
 - ❖ Requires little memory
 - ❖ often find reasonable solutions in large or infinite state spaces for which systematic algorithms are unsuitable.



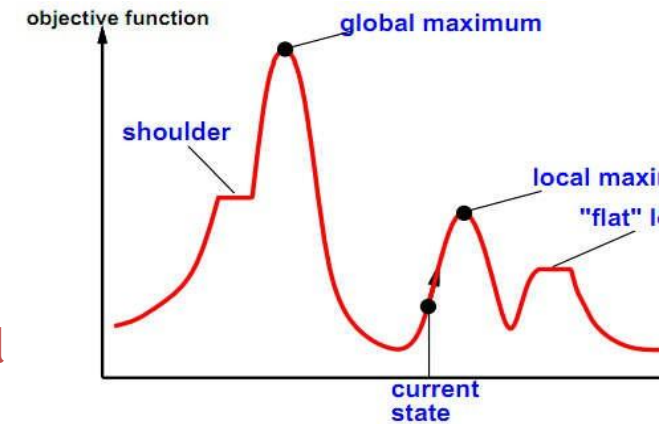
Local Search: Key Idea

Advantages	Drawback:
• Use very little memory - usually a constant amount. • Can often find reasonable solutions in large or infinite state spaces (e.g., continuous). For which systematic search is unsuitable.	• Local Search can get stuck in local maxima and not find the optimal solution.

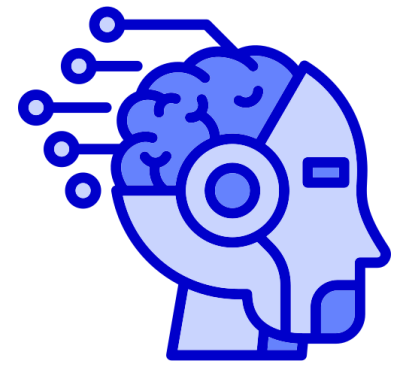
State-Space Landscape



- A state space landscape: is a graph of states associated with their costs.
- State-space landscape
 - Location (defined by state)
 - Elevation (defined by the value of the heuristic cost function or objective function)
 - If elevation = cost, aim to find the lowest valley (a **global minimum**)
 - If elevation = objective function, find the highest peak (a **global maximum**)
 - A complete local search algorithm always find a goal if one exists
 - An **optimal** algorithm always find a **global minimum/maximum**

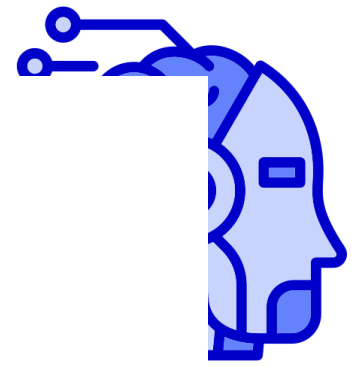


Local and Global Optima



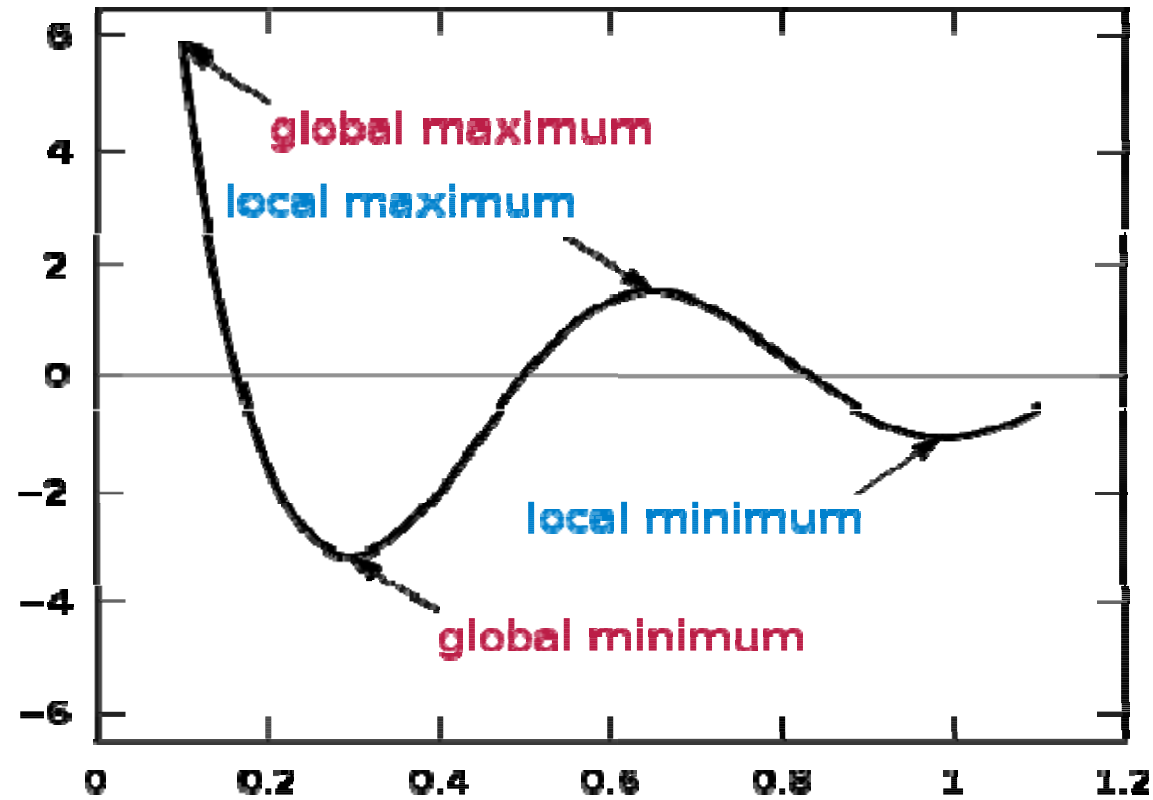
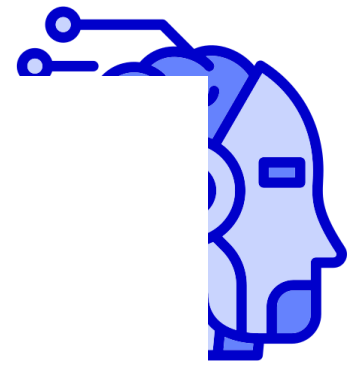
- Global optimum
 - A solution that is better than all other solutions
 - Or no worse than any other solution
- Local optimum
 - A solution which is better than nearby solutions
 - A local optimum is not necessarily a global one

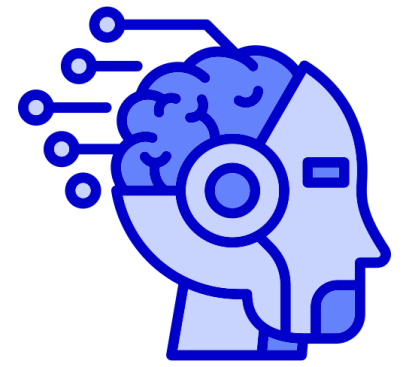
Global /Local(max/min)



- A local max/min is over a small area.
 - For instance, if a point is lower than the next nearest point on the left & right then it's a local min.
- There can be many local maxes and mins over an entire graph.
- A global max/min is the highest/lowest point on the entire graph.
- There can only be ONE global max and/or min on a graph and there may not be one at all.

Global /Local(max/min)

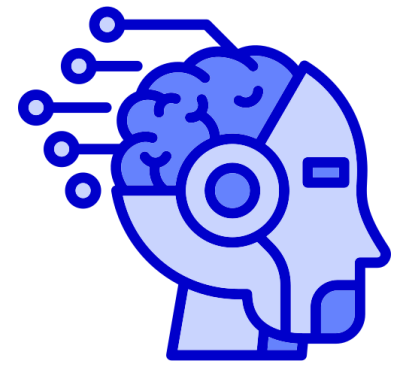




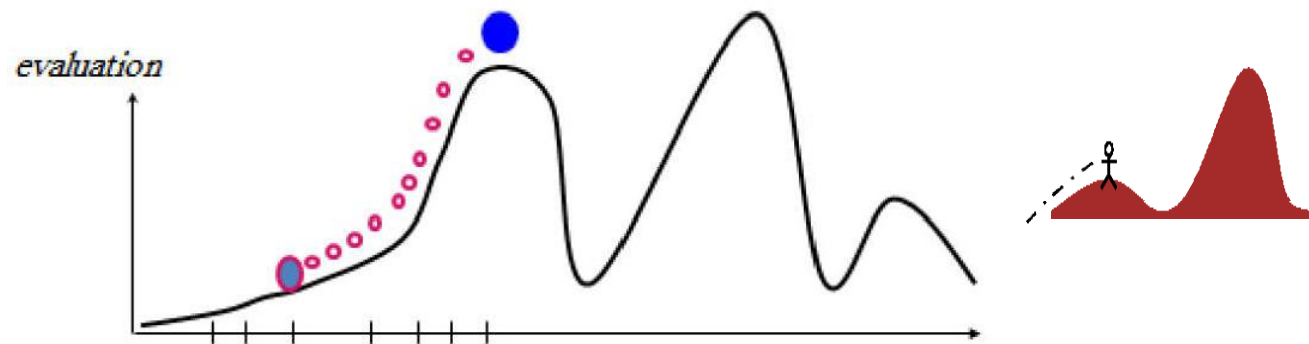
Hill-Climbing Search

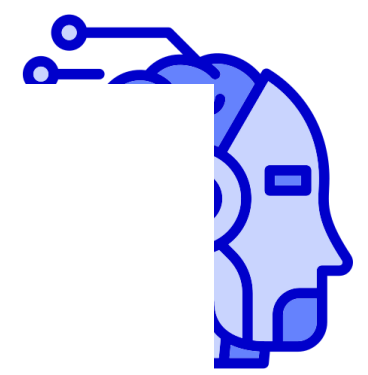
Artificial Intelligence

Hi I-Climbing Search



- **Main Idea:** Keep a single current node and can move to a neighboring state to improve it.
- Uses a loop that continuously moves in the direction of increasing value (**uphill**):
 - Choose the best successor, choose **randomly** if there is more than one.
 - Terminate when a peak is reached where no neighbor has a higher value.
- It is also called greedy local search, steepest ascent/descent.





Hill-Climbing Search

- “Like climbing Everest in thick fog with amnesia”
- Only record the state and its evaluation instead of maintaining a search tree

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

inputs: *problem*, aproblem

local variables: *current*, anode
 neighbor, anode

current $\leftarrow$ MAKE-NODE(INITIAL-STATE[*problem*])

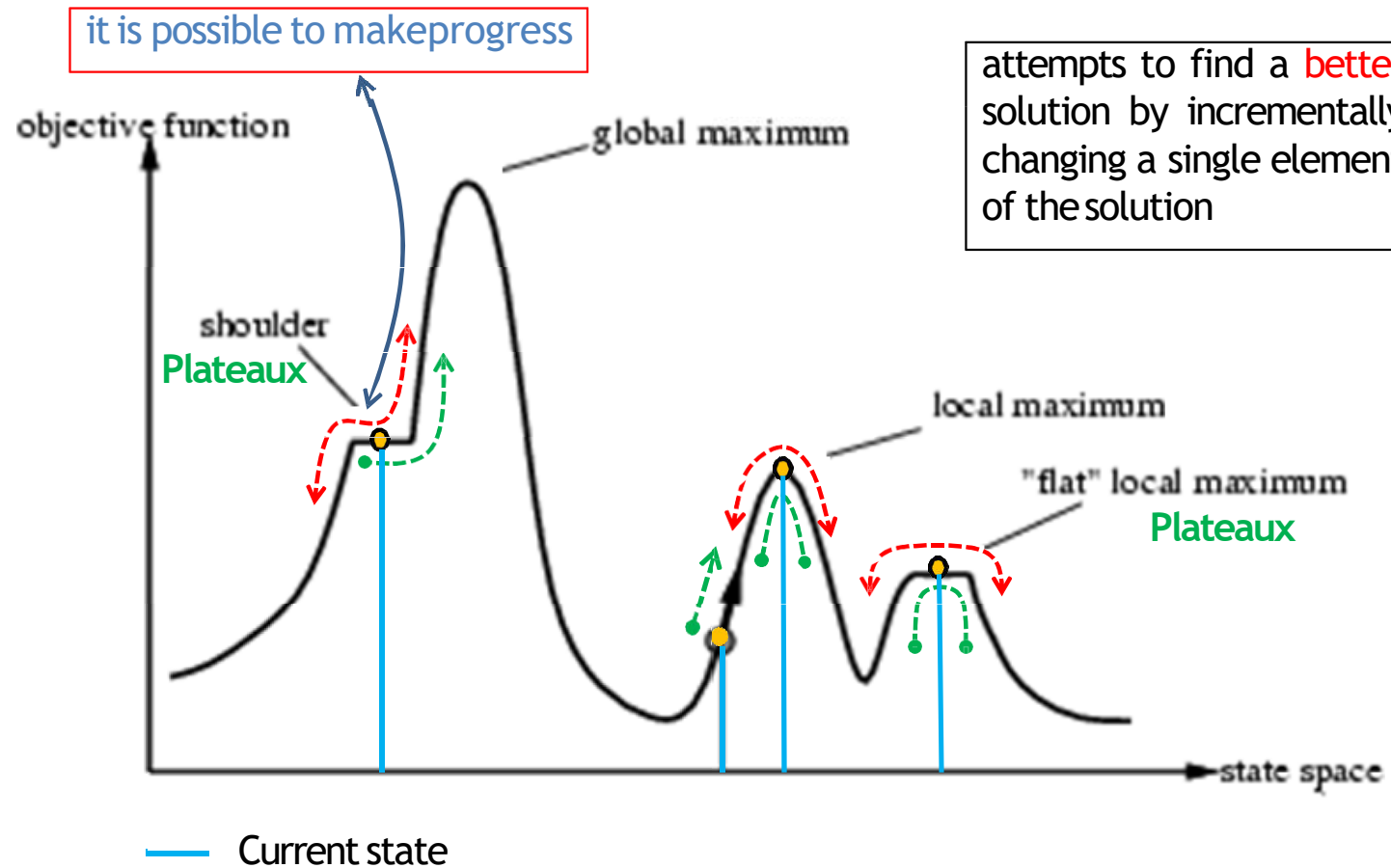
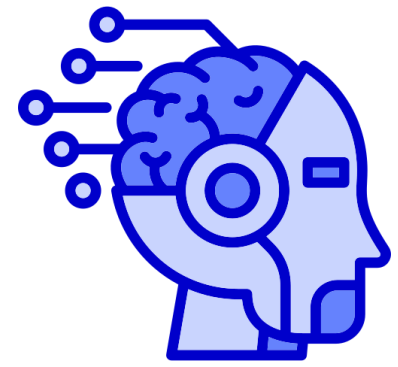
loop do

neighbor $\leftarrow$ a highest-valued successor of *current*

if VALUE[*neighbor*] $\leq$ VALUE[*current*] **then return** STATE[*current*]

current $\leftarrow$ *neighbor*

Hi I-Climbing Search

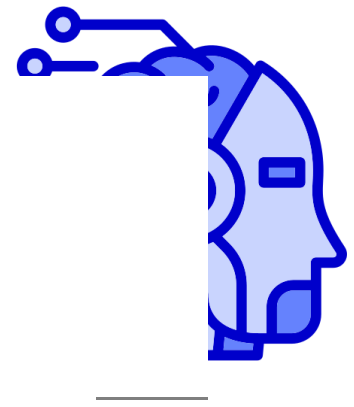
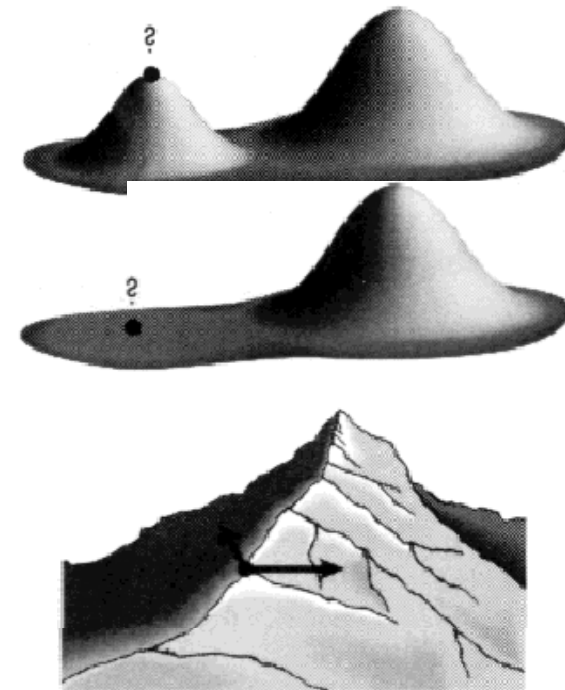


Hill-Climbing Search

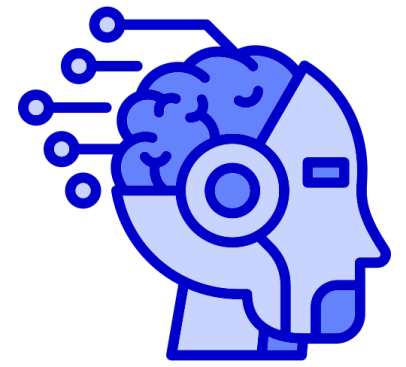
Local maxima: a local maximum is a **peak that is higher than each of its neighboring states, but lower than the global maximum**. Hill-climbing algorithms that reach the vicinity of a local maximum will be drawn upwards towards the peak, but will then be stuck with nowhere else to go.

Plateaux: a plateau **is an area of the state space where the evaluation function is flat**. It can be a flat local maximum, from which no uphill exit exists, or a **shoulder**, from which it is possible to make progress.

Ridges: Ridges result in a sequence of local maxima that is very difficult for greedy algorithms to navigate. **(the search direction is not towards the top but the side)**

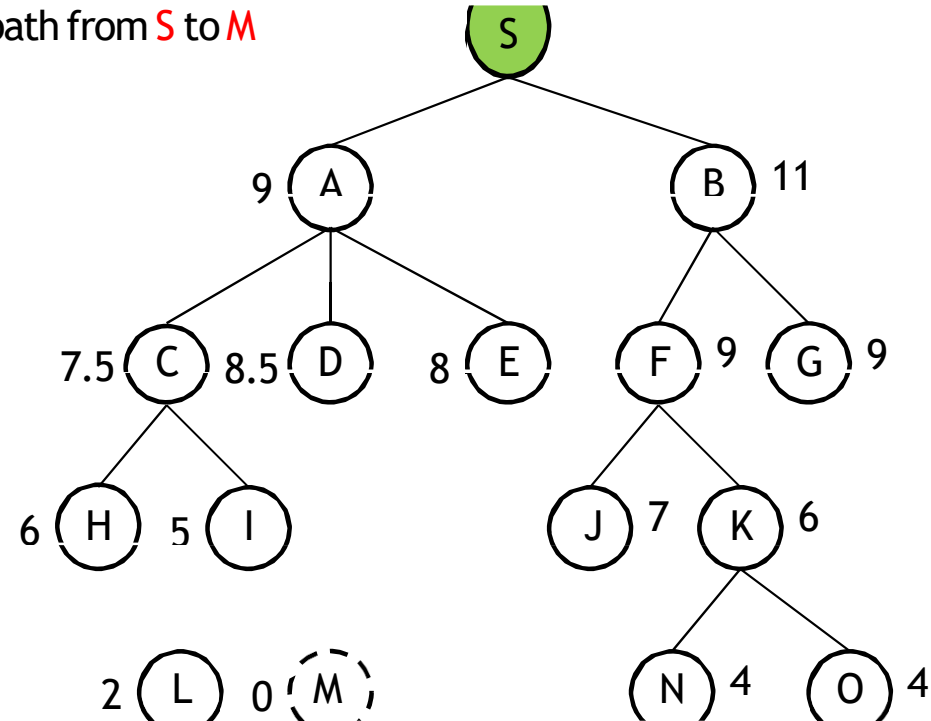


Hill-Climbing Search



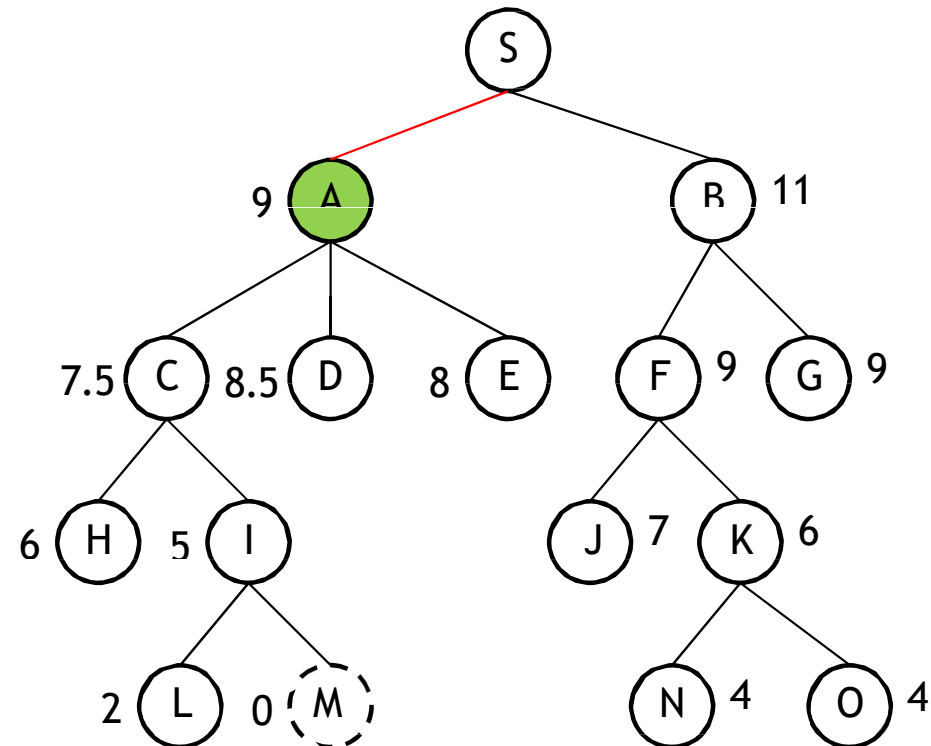
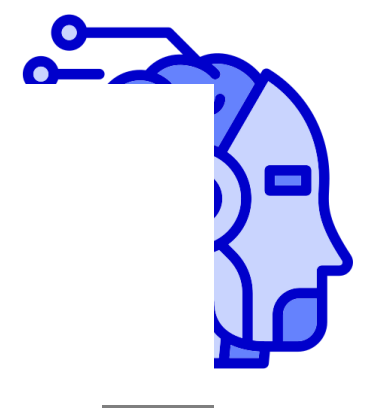
a

our aim is to find a path from **S** to **M**

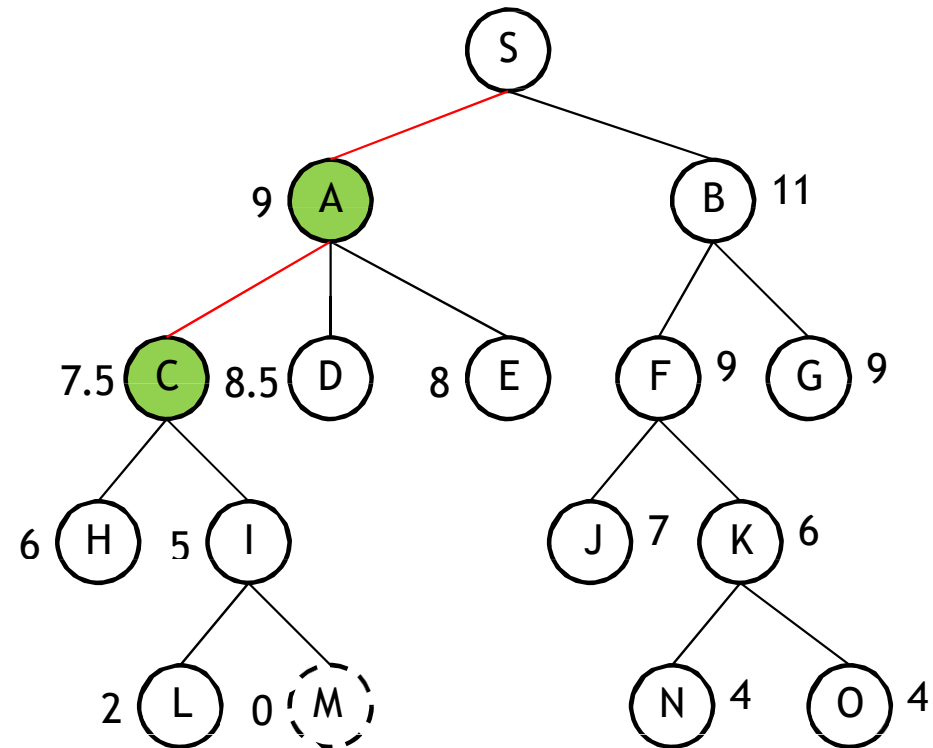
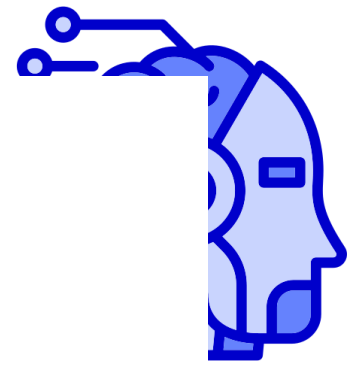


associate
heuristics with
every node,
that is the
straight line
distance
from the path
terminating
city to the goal city

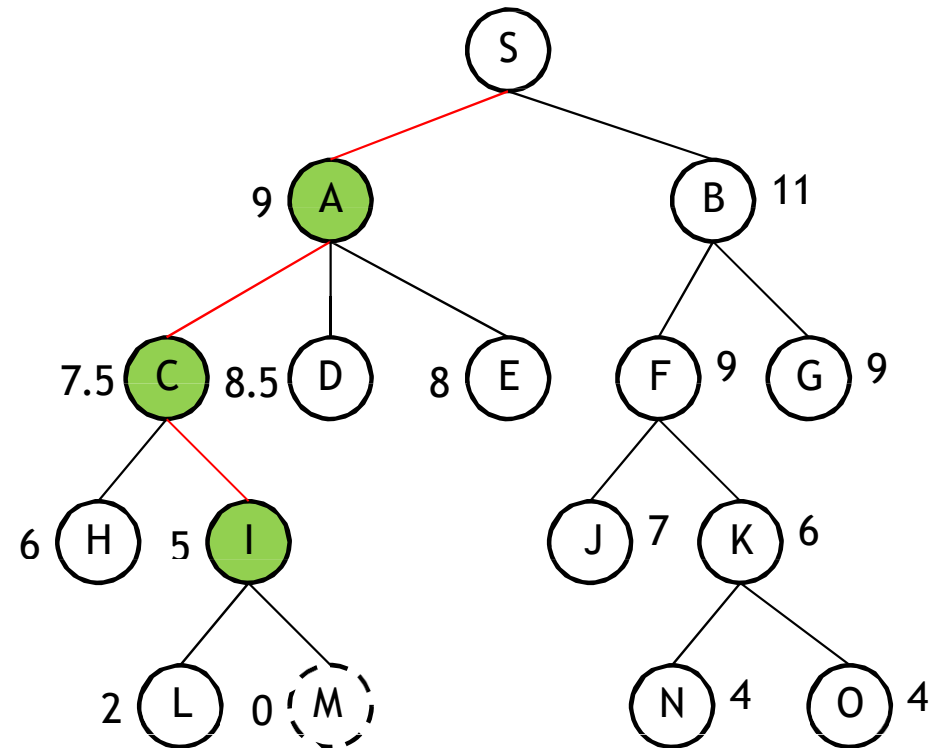
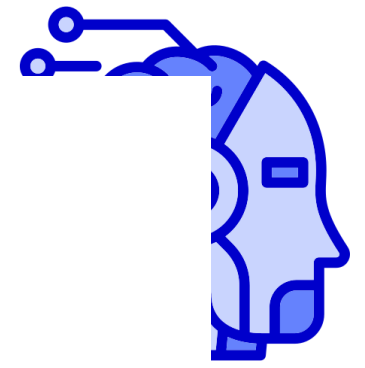
Hill-Climbing Search



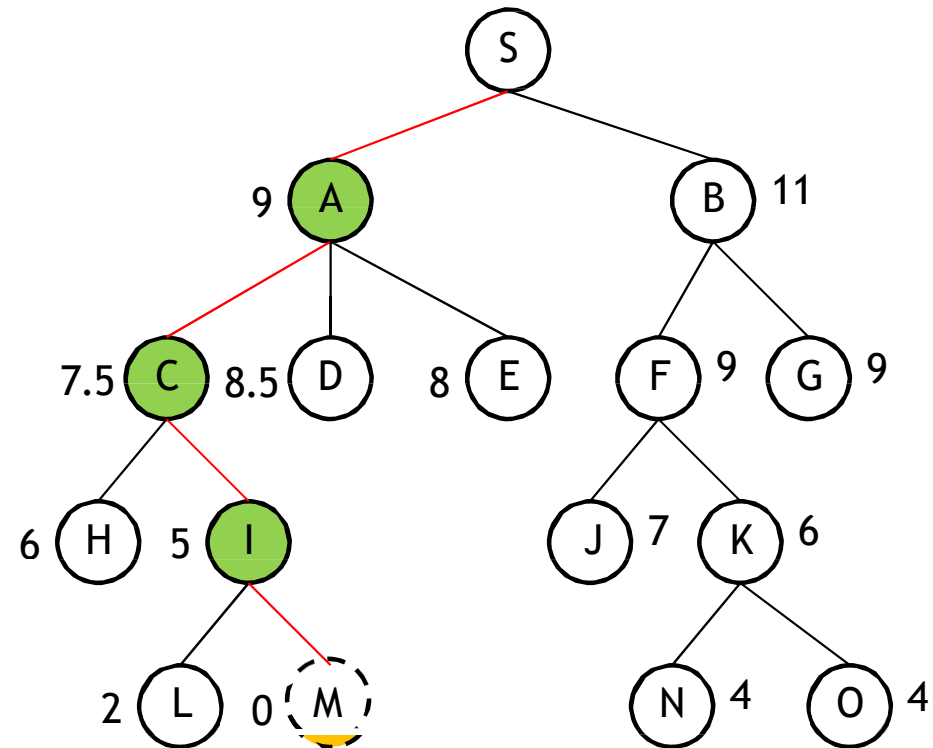
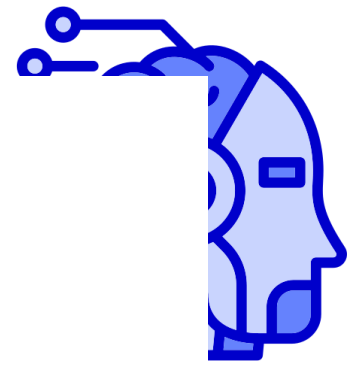
Hill-Climbing Search



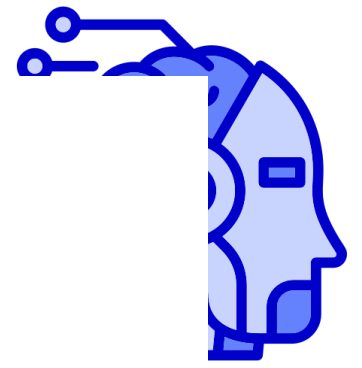
Hill-Climbing Search



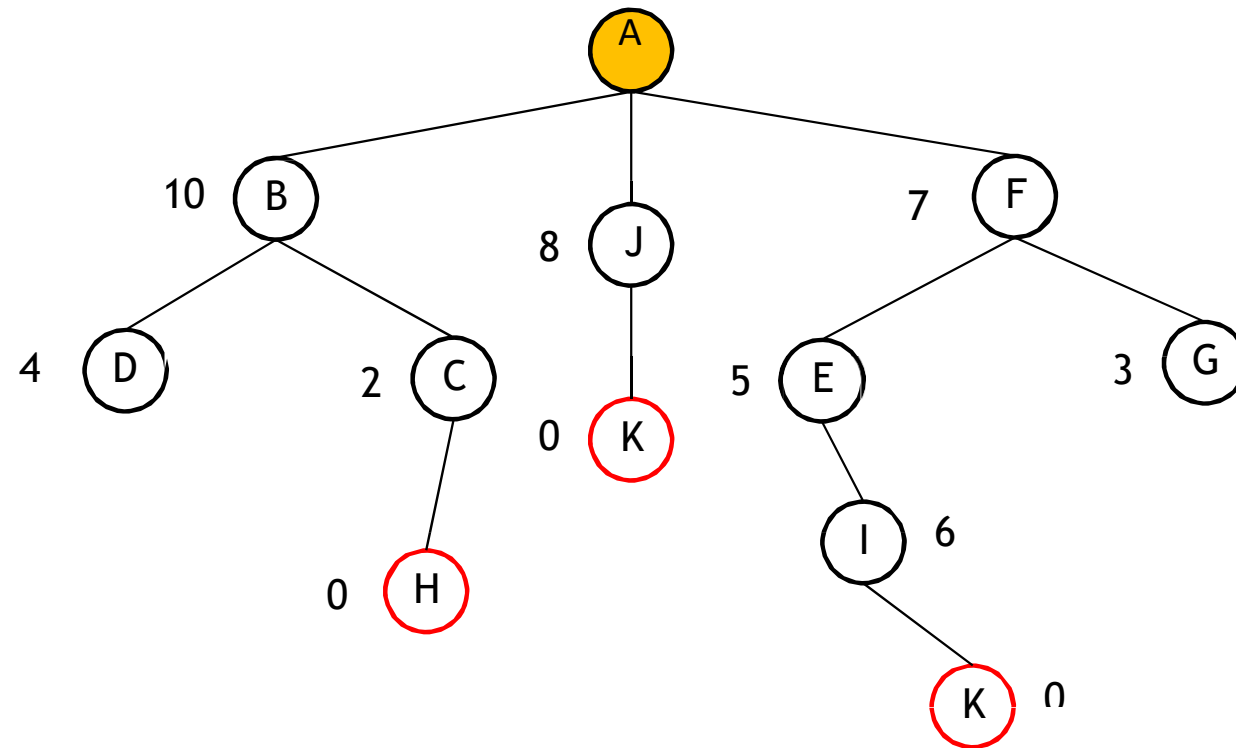
Hill-Climbing Search

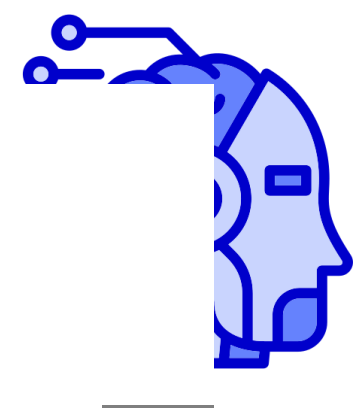


Hill-Climbing Search Example

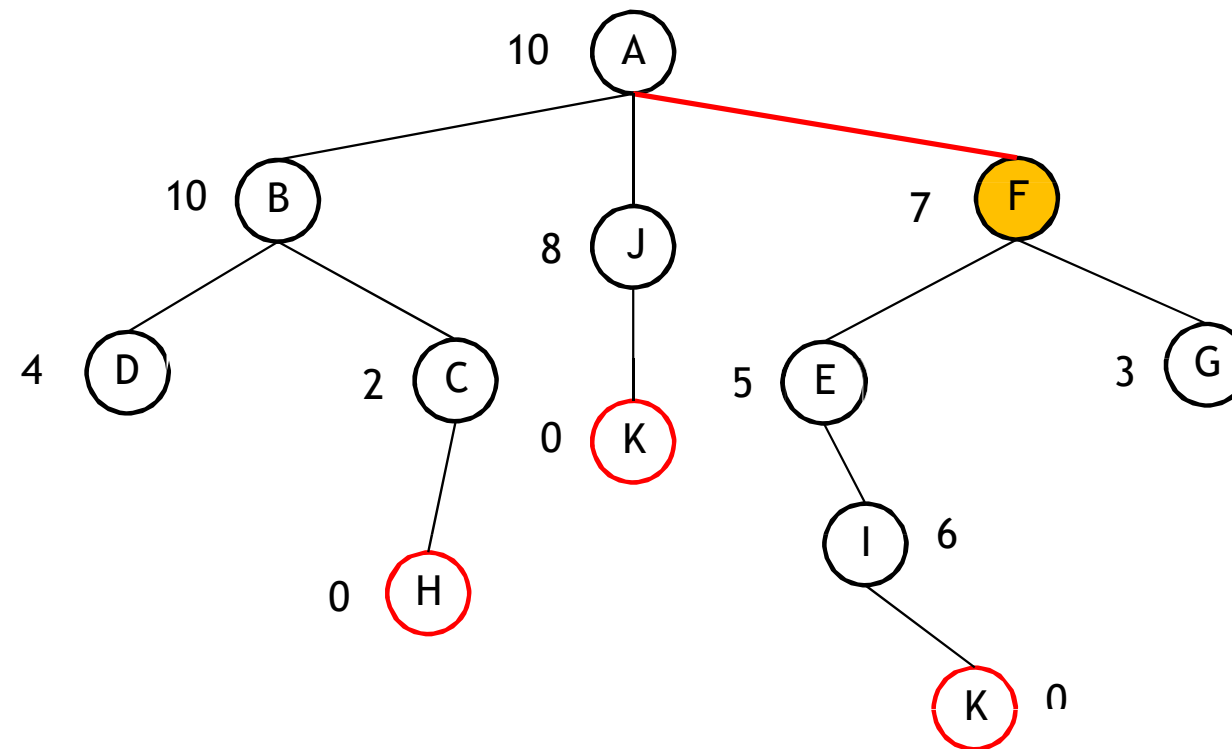


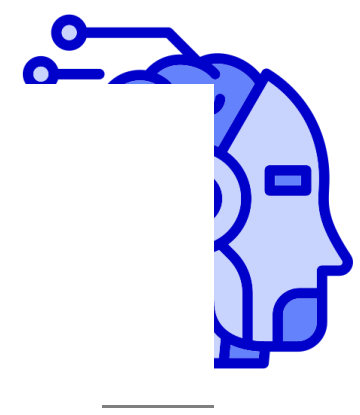
Start Node= A
Goal Nodes = H & K



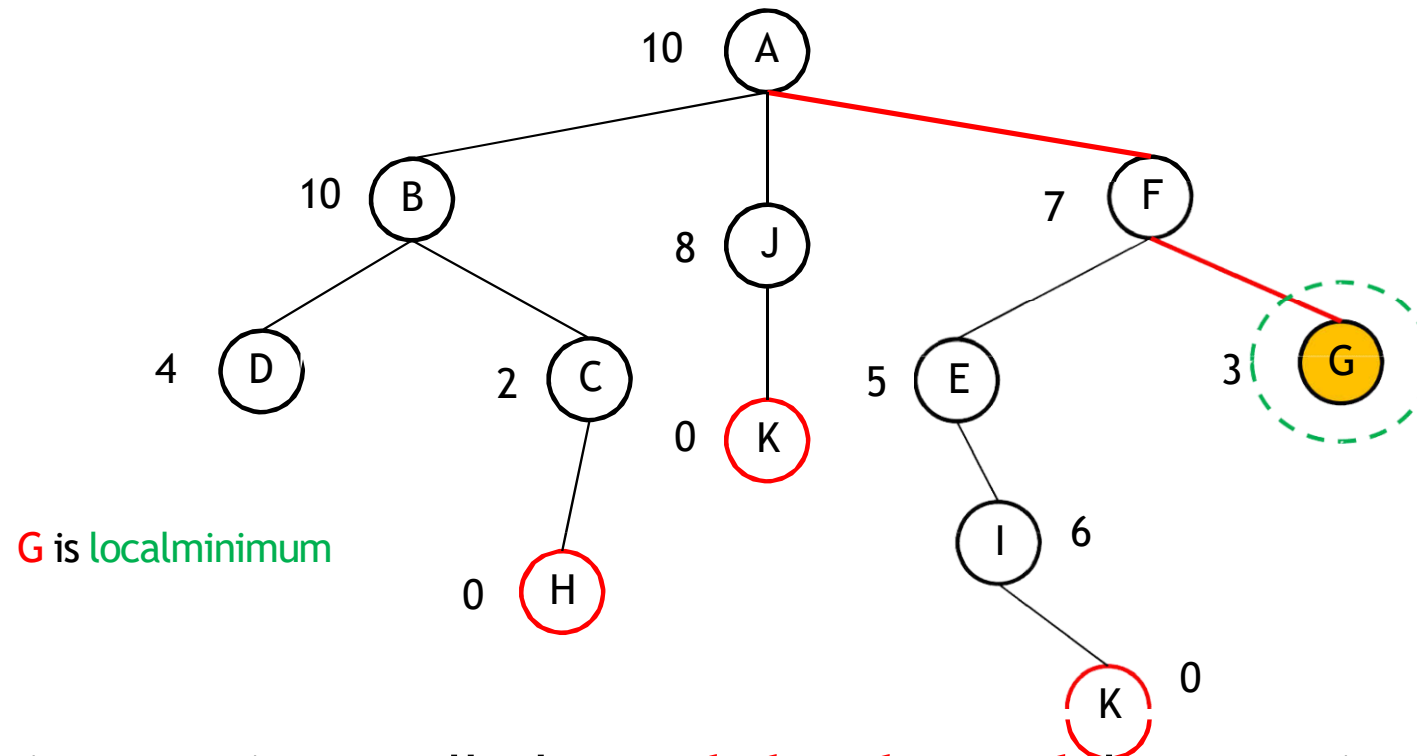


Local Maximum

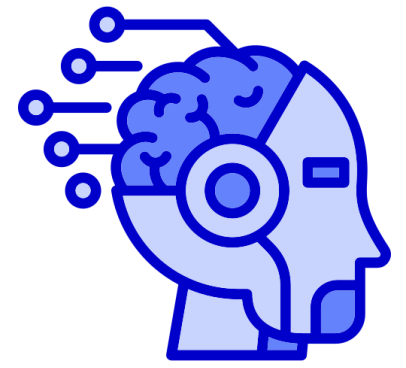




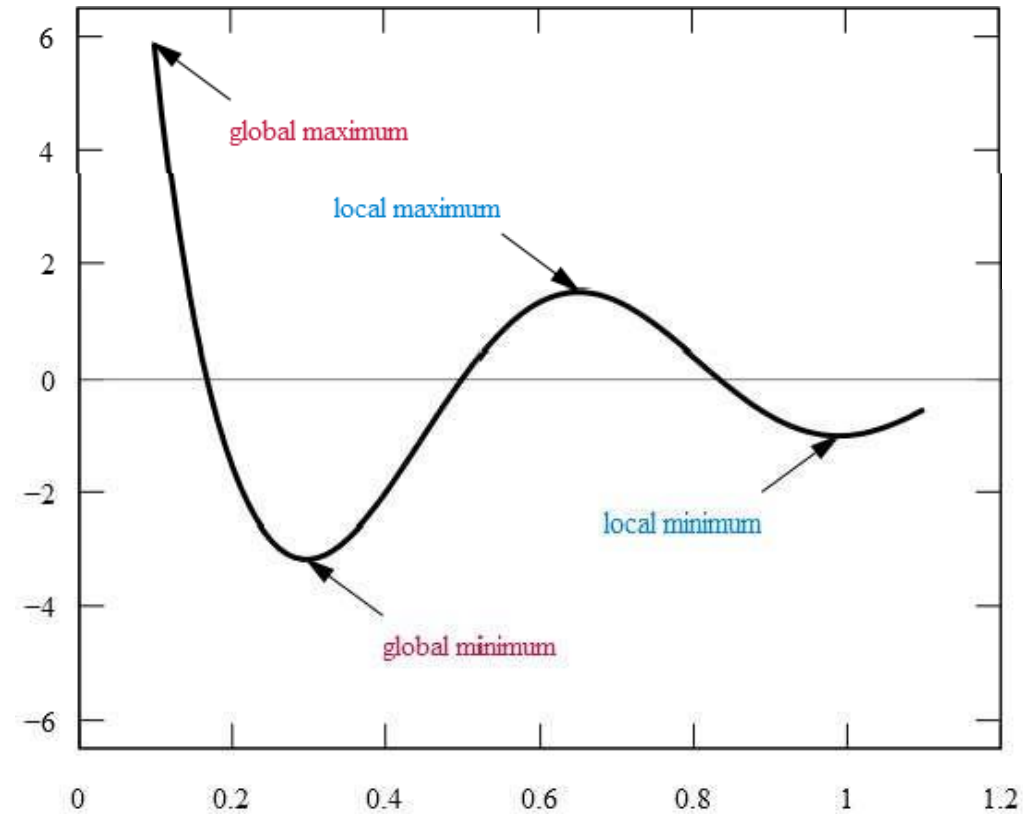
Local Minimum



Hill Climbing is sometimes called **greedy local search** because it grabs a good neighbor state without thinking a head about



Local Maximum, Local Minimum



Artificial Intelligence